

CRUD Operations & Routing (Theory)

Welcome to CRUD Operations & Routing! As your backend grows, putting all your API routes and database queries into a single `main.py` file becomes difficult to maintain. In this session, you will learn how to structure scalable applications using FastAPI's `APIRouter` and master the principles of **CRUD (Create, Read, Update, Delete)** operations.

1. Structuring Large Applications with `APIRouter`

As backend projects expand, organization is crucial to maintain readability, testability, and team collaboration.

A. The Monolith Problem vs. Modular Design

- **Monolithic Approach:** Putting 50+ endpoints, database models, and schemas in one file creates a cluttered codebase that is hard to debug.
- **Modular Approach:** Grouping related functionality into separate modules (e.g., authentication routes, prediction routes, user management) using routers.

B. What is `APIRouter` ?

`APIRouter` is a class in FastAPI that allows you to create mini-routers (sub-applications) that can be mounted onto your main FastAPI instance.

- **How it Works:** You define routes in separate files (e.g., `routers/predictions.py`) using an `APIRouter` instance, and then include them in `main.py` with a shared prefix (`/api/v1`).

Conceptual Example:

```
# routers/predictions.py
from fastapi import APIRouter

router = APIRouter(prefix="/predictions", tags=["Predictions"])

@router.get("/")
```

```
def get_predictions():
    return [{"id": 1, "result": 1500.0}]
# main.py
from fastapi import FastAPI
from routers import predictions

app = FastAPI()
app.include_router(predictions.router)
```

2. The CRUD Paradigm in REST APIs

CRUD represents the four basic functions of persistent storage. In RESTful web design, these operations map directly to HTTP methods and database actions:

Operation	HTTP Method	CRUD Action	SQL Equivalent	FastAPI Purpose
Create	POST	Insert new record	INSERT INTO	Submit new features for ML inference & log result
Read	GET	Retrieve record(s)	SELECT	Fetch past prediction logs or specific records
Update	PUT / PATCH	Modify existing record	UPDATE	Edit existing log metadata or labels
Delete	DELETE	Remove a record	DELETE	Delete invalid or old prediction logs

3. Designing Robust CRUD Endpoints

When implementing CRUD operations in FastAPI with SQLAlchemy, best practices ensure data integrity and security:

A. Input and Output Schemas (`response_model`)

Never expose your raw database models directly to the client. Instead, use separate Pydantic schemas:

- Input Schemas:** Validate incoming request bodies (`CreatePrediction` , `UpdatePrediction`).
- Output Schemas (`response_model`):** Filter out sensitive fields (like internal database IDs or credentials) and format outgoing JSON structures cleanly.

B. Error Handling & Status Codes

Proper error handling ensures clients receive meaningful feedback when operations fail:

- `404 Not Found` : When querying an ID that does not exist in the database.
 - `400 Bad Request` : When update parameters violate business rules.
 - `500 Internal Server Error` : Unhandled database transaction failures (handled safely via rollbacks).
-

4. Reference Links & Further Reading

- **FastAPI Bigger Applications - Multiple Files**: Official guide on structuring projects with `APIRouter`.
 - **FastAPI Response Model Documentation**: Learn how to use Pydantic models to filter and format API responses.
-